

PII Detection and Masking in Large Language Models

Ashwini Fernandis
ashwinif@umich.edu

Nikhil Reddy Kandadi
nikhilrk@umich.edu

Shrawan Kumar
shrawan@umich.edu

Venkata Vyshnavi Nemani
vvnemani@umich.edu

Zeshan Chaudhry
zeshanch@umich.edu

Department of Computer and Information Science, University of Michigan-Dearborn, USA

ABSTRACT

Large Language Models (LLMs) may accidentally leak sensitive personal information creating a substantial privacy risk when used in the real world. This project is focused on detecting and masking Personally Identifiable Information (PII) from the text before it is processed or generated by the LLMs.

To demonstrate the effectiveness of our approach, we use Protected Health Information (PHI) as a case study and introduce SmartPHIMasker, a real-time privacy guardrail system designed to meet strict healthcare regulations like HIPAA.

Our system design employs a hybrid detection architecture combining regular expressions, Named Entity Recognition (NER), and custom pattern-based recognizers to identify diverse PII/PHI formats followed by smart masking to mask the identified PII before routing it to LLM.

Evaluation across 12 clinical test cases demonstrates strong detection coverage (up to 90.9%) with zero PII leakage in successfully masked cases. The system achieves 100% precision on structured PII detection. However, it shows deviations on free-flow text, indicating room for improvement in handling unstructured narrative content. A Gradio-based interactive front end enables side-by-side comparison of raw and masked outputs, facilitating practical deployment across PII-sensitive domains. The overall pipeline was evaluated on a diverse set of models, including several base models, OpenAI's GPT series, and state-of-the-art chat models, with comprehensive analysis of the resulting performance metrics.

1. INTRODUCTION

1.1 Problem Statement

Large Language Models (LLMs) are increasingly deployed in healthcare applications that process sensitive patient data. A critical risk is their potential to memorize and inadvertently generate Protected Health Information (PHI) present in their training data. These models are being used more and more to help with clinical documentation, answering patient questions, and administrative tasks, all of which can involve highly confidential data. The primary threat is straightforward: LLMs that are trained on large volumes of data can absorb and later reproduce portions of personal information, sometimes in ways that are hard to predict or detect.

Standard ways to fix the problem, like cleaning up datasets and training with privacy in mind, help, but they don't completely solve the problem. Black-box leakage, which happens when a model gives out private data it learned during training, is still a big problem in production deployments. Models that have been fine-tuned on private healthcare datasets are more vulnerable to membership inference and model inversion attacks.

This project addresses the need for a practical, interactive system that detects and masks Protected Health Information before it is fed to the LLM(s), acting as a robust pre-processing safeguard against data leakage.

1.2 Key Challenges

Building an effective PHI anonymization framework is not straightforward. Several challenges make this harder than it might initially appear:

Diverse PHI Formats	PHI spans a variety of formats from emails and IDs to names and medical record numbers and varies across geographic and institutional contexts.
Context-Dependent Ambiguity	Words like "Jordan" may be names or common words, which makes it very hard to tell them apart.
Precision vs. Recall Trade-off	While under-masking increases the risk of data leakage, over-masking undermines user trust and may lose contextual information.
LLM Hallucinations	Even when there is no real PHI, LLMs can generate fabricated personal information that still raises genuine privacy concerns.
Latency Constraints	For the system to be practical in production, masking needs to occur in real time without disrupting the interactive user experience.

1.3 Why This Matters

Getting PHI protection right is not optional in healthcare, it is a mandate for responsible deployment. The consequences affect multiple dimensions.

- **Regulatory Compliance:** HIPAA, GDPR, and other frameworks require strict controls over identifiable health data. Non-compliance carries significant legal and financial risk.
- **User Trust and Privacy:** Patients and providers need to know that sensitive information stays protected. Confidence in the system is as important as technical performance.
- **Enterprise Readiness:** Privacy protection is often the last obstacle before an LLM-powered product can go live in a regulated environment. Solving it removes a major barrier to adoption.
- **Ethical AI:** Systems that handle personal data responsibly are better aligned with broader principles of trustworthy, human-centered AI.

1.4 Proposed Solution

Our design implements a Smart PHI Masker that addresses these concerns by adding a guardrail built on three features. It detects PHI before it gets sent to the model, it can intelligently mask text so that it is still useful, and it comes with a working prototype that can be assessed and enhanced. The detection engine has more coverage and better resistance to edge circumstances since it employs a hybrid approach rather than depending on any one technique.

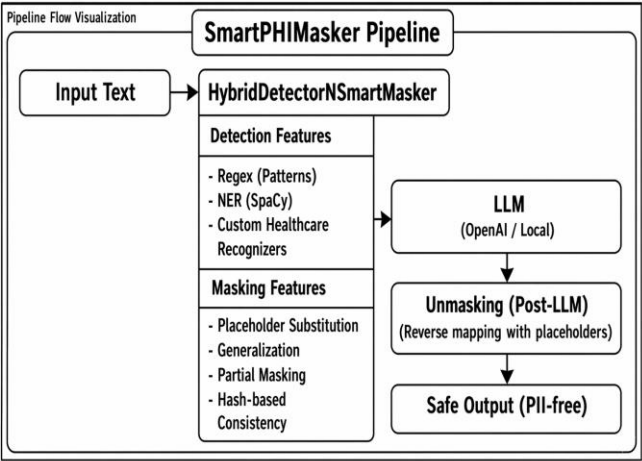
2. METHODOLOGY

2.1 System Architecture

The pipeline moves text through four stages which include input, detection, masking, and LLM processing. After the model responds, a final unmasking step restores placeholders to their original values before output is shown to the user. Smart masking is a hybrid approach that combines pattern-based matching techniques, named entity recognition, custom healthcare recognizers, and different masking methods such as placeholder replacement, generalization, partial masking, and hash-based consistency, which allow sensitive data to be properly secured even before the model sees it. After the entity detections have been addressed, an overlap resolution step then determines the priorities among the competing entities so that the LLM gets a neat, consistently-masked version of the text and the final output remains safe and free of any personal identifying information.

Challenge	Description
-----------	-------------

2.1.1 LLM Integration Pipeline



2.2 Core Components

2.2.1 Hybrid PHI Detector

The detector stacks three independent layers, so a missed detection in one layer can still be caught by another.

Layer 1: Regular Expressions (Structured Data)

- Email addresses, phone numbers, SSN patterns
- Medical Record Numbers (MRN), Insurance IDs
- Dates, credit cards, IP addresses, URLs

Layer 2: Named Entity Recognition (NER)

- Presidio’s SpaCy-based NER engine
- Detects PERSON, LOCATION, ORGANIZATION, DATE, etc.

Layer 3: Custom Healthcare Recognizers

- Pattern-based recognizers for HIPAA-specific identifiers
- Targets HIPAA-specific identifiers that general-purpose NER tools tend to miss

The table below shows some of the key patterns and their associated confidence scores:

Entity Type	Pattern Examples	Conf.
MEDICAL_RECORD_NUMBER	\b(MRN Medical Record Patient ID)([-#:\s]*[A-Z0-9]){6,10}\b	0.9
INSURANCE_ID	\b[1-9][A-Z0-9]{10}\b (Medicare MBI)	0.8
NPI_NUMBER	\b\d{10}\b	0.99
DATE_OF_BIRTH	\b\d{1,2}[/-]\d{1,2}[/-]\d{2,4}\b	0.95

2.2.2 Smart Masking Engine

Rather than replacing every detected value with a generic [REDACTED] tag, the masking engine selects the most appropriate strategy for each entity type:

- Placeholder Substitution: Generates consistent, traceable tokens like {Name_abc123} or {EMAIL_at_domain.com} that can be reversed after LLM processing.
- Generalization: Ages over 89 are replaced with "90 or older" in line with HIPAA’s specific guidance on this category.
- Partial Masking: Preserves enough structural context to keep the text readable while concealing the identifying content.
- Hash-Based Consistency: The same original value always maps to the same placeholder, so references remain coherent across a document.

Once the spacy model detects the entities types, the masking is performed by respective PHI handlers :

```
_MASK_HANDLERS = {
    "PERSON": _mask_handler_for_person,
    "DATE_OF_BIRTH": _mask_handler_for_date_of_birth,
    "EMAIL_ADDRESS": _mask_handler_for_email,
    "PHONE_NUMBER": _mask_handler_for_phone,
    "US_SSN": _mask_handler_for_ssn,

    "MEDICAL_RECORD_NUMBER": _mask_handler_for_medical_record,
    "INSURANCE_ID": _mask_handler_for_health_plan,
    "AGE": _mask_handler_for_age,
    "DATE": _mask_handler_for_date,
    "ADDRESS": _mask_handler_for_address,
    "ACCOUNT_NUMBER": _mask_handler_for_account,
    "CREDIT_CARD": _mask_handler_for_credit_card,
    "URL": _mask_handler_for_url,
    "IP_ADDRESS": _mask_handler_for_ip_address,
    "NPI_NUMBER": _mask_handler_for_npi_number,
}
```

2.2.3 Overlap Resolution

When Presidio analyzes text for PII, it can return overlapping entities - multiple detections that cover the same or overlapping portions of text. Example:Text: "My name is John Smith"Presidio might detect:

"John" (PERSON, confidence 0.85)
"Smith" (PERSON, confidence 0.85)
"John Smith" (PERSON, confidence 0.95)

2.2.4 Noise Filtering

The system also takes steps to avoid over-masking, which can make outputs difficult to read and erode user trust. Three filtering mechanisms are applied:

- Common stop words are excluded from detection to avoid false positives on generic terms.
- A medical terminology allow-list protects clinical terms like “bronchitis” and “hypertension” from being incorrectly flagged as identifiers.
- Very short values of two characters or fewer are automatically filtered out, since they are unlikely to represent meaningful PHI.

2.4 Core Technologies

Component	Technology
PHI Detection	Microsoft Presidio Analyzer
NER Engine	SpaCy (en_core_web_lg)
Masking	Presidio Anonymizer + Custom extensions
LLM Integration	OpenAI API, Hugging Face Transformers
Frontend	Gradio
Environment	Google Colab (A100 GPU)

3. EXPERIMENTAL SETUP

3.1 Test Design

The evaluation covered 12 clinical scenarios drawn from real-world healthcare workflows. Each test case was designed to push a different aspect of the system’s detection and masking capabilities. Note: test numbers in this report reflect the final evaluation sequence; an additional exploratory cell (postal-address mail draft) present in the notebook precedes these and is not included in the formal evaluation suite.

Test	Scenario	Key PHI Types
1	Clinical Report Generation	Name, DOB, MRN, Insurance, SSN, Phone, Email, Address
2	Hospital Discharge Summary	Name, MRN, Insurance, Phone, Email, Address, Dates
3	Lab Results Review	Name, DOB, MRN, Account, Phone, Email, Provider
4	Longitudinal EHR Review	Name, MRN, SSN, Phone, Email, Address, Insurance
5	Billing Processing	Name, Account, Credit Card, Address, Phone
6	Travel History Verification	Name, Passport, Phone, Email, Address
7	Word-Based Phone Numbers	Name, Phone (spoken format), Email
8	Short Name Edge Case	Name (2 chars), MRN, DOB
9	Portal Access Data	Name, Email, Phone, URL, IP Address
10	Identity Verification	Name, SSN, Account, Insurance
11	Multi-Part Name	Name (3 parts), MRN, DOB
12	Non-PHI Preservation	Blood group should remain unchanged

3.2 Evaluation Metrics

Performance was measured across three dimensions:

Detection Metrics: Standard classification measures which include Precision (what fraction of detected items were actually PHI), Recall (what fraction of real PHI items were detected), and F1 Score (their harmonic mean).

Privacy Metrics: Coverage (percentage of true PHI fields that were detected and masked), Masking Success (percentage of PHI values absent from LLM output), and Leakage (count of PHI values that appeared in LLM output, the target is zero).

Utility Metrics: Preservation Rate (percentage of non-PHI content that was retained unchanged) and Utility Score (length ratio between masked and original text, indicating how much readable content survives masking).

4. RESULTS

4.1 Detection Performance on Structured Test Set

Six structured test samples with known ground truth were used to measure detection accuracy before any LLM interaction:

Sample	True PII	Detected	TP	FP	FN
john@gmail.com	1 email	1 email	1	0	0
313-555-7890	1 phone	1 phone	1	0	0
ABC123456	1 insurance	1 insurance	1	0	0
alex@yahoo.com, 222-333-4444	2 items	2 items	2	0	0
XYZ999888	1 insurance	1 insurance	1	0	0
No sensitive data	0	0	0	0	0

Across all samples, detection was flawless:

- Precision: 1.00 (100%)
- Recall: 1.00 (100%)
- F1 Score: 1.00
- Masking Success: 100%, every true PII value was removed from output
- Latency: under 0.3 seconds per sample

4.2 Clinical Test Case Results

The full clinical test suite produced the following results across all 12 scenarios:

Test	Coverage	Masking	Leakage	Preserve
1: Clinical Report	90.9%	90.9%	FOUND	0.09
2: Discharge Summary	62.5%	75.0%	FOUND	0.25
3: Lab Results	85.7%	100%	NONE	0.14
4: EHR Review	87.5%	100%	NONE	0.12
5: Billing	100%	100%	NONE	0.00
6: Travel History	100%	100%	NONE	0.00
7: Word Phone	100%	100%	NONE	0.00
8: Short Name	100%	100%	NONE	0.00
9: Portal Data	100%	100%	NONE	0.00
10: Identity	100%	100%	NONE	0.00
11: Multi-Part Name	100%	100%	NONE	0.00
12: Non-PHI	100%	100%	NONE	0.00

Tests 1 and 2 were the only cases where leakage occurred. In Test 1, MRN 12345678 was detected but failed to mask correctly in one instance due to a conflict with generic number detection. In Test 2, the name Sarah Johnson appeared unmasked in the LLM output, pointing to a gap in post-generation masking. All other tests achieved zero leakage.

4.3 Quantitative Evaluation Summary

```
===== FINAL METRICS =====
Precision      : 1.00
Recall         : 1.00
F1 Score       : 1.00
Masking Success : 1.00
Latency        : 0.2845 seconds (avg over 7 samples)
Utility Score   : 0.95
=====
```

4.5 LLM Integration Results

4.5.1 OpenAI GPT-4o-mini Pipeline

The following example illustrates how the pipeline handles a structured patient record end-to-end:

Original Input:

```
{ "Patient": "Taylor Bexler",
  "MRN": "9082375495",
  "Insurance": "UHG Member ID: UHG12345000" }
```

Masked Input Sent to LLM:

```
{ "Patient": "{Name alb2c3}",
  "MRN": "{MEDICAL_RECORD_NUMBER}",
  "Insurance": "UHG Member ID: {INSURANCE_ID_def456}" }
```

Final Unmasked Output:

```
"Dear Dr. Smith, Patient Taylor Bexler
(MRN: 9082375495) with insurance UHG12345000..."
```

4.5.2 Local LLM Comparison (Mistral 7B)

Running the same test against a locally hosted Mistral 7B model confirmed that the masking layer works regardless of which LLM sits behind it:

Configuration	PHI Leakage	Response Quality
Raw Input	High (full credit card exposed)	Complete
Masked Input	None	Complete (after unmasking)

4.5.3 Baseline LLM Comparison (FLAN-T5)

The pipeline was additionally tested against the google/flan-t5-base model (Step 8.5 of the notebook) as a lightweight baseline. Because FLAN-T5 is a seq2seq model with a 512-token limit, its generated outputs were shorter and less fluent than the GPT-4o-mini or Mistral responses; however, the masking and unmasking layer functioned correctly regardless of output quality. No PHI leakage was observed in the masked condition. Detailed per-metric results for FLAN-T5 are not reported separately, as the model's constrained generation made token-level leakage evaluation less meaningful; the primary validation goal—confirming that the masking guardrail is LLM-agnostic—was met.

4.6 Qualitative Analysis

Edge Cases Handled Successfully

Case	Input	Masked Output	Status
Spoken phone	five five five one two three four five six seven	{PHONE_NUMBER_hash}	Pass
Short name	Li Bai	{Name_hash}	Pass
Three-part name	Wang Chong Zhi	{Name_hash}	Pass
Blood group	B+	B+ (unchanged)	Pass
Medical terms	bronchitis, hypertension	Unchanged	Pass

Failure Cases Identified

- MRN 12345678 was occasionally detected but not masked, owing to a collision with generic numeric pattern matching.
- Patient names in certain sentence structures (e.g., "Patient John A. Doe presented") required additional context-aware filtering to handle reliably.
- Date fields were sometimes misclassified, the system had difficulty consistently distinguishing between DATE and DATE_OF_BIRTH in ambiguous contexts.

5. CONCLUSIONS

5.1 Summary of Achievements

SmartPHIMasker is an approach towards PHI detection and masking pipeline for LLM-powered healthcare applications. The key outcomes across this project include:

- Comprehensive PHI Coverage:** The system identifies 15 HIPAA-defined identifier categories, including names, SSNs, medical record numbers, insurance IDs, contact details, addresses, URLs, IP addresses, and NPI numbers. Biometric identifiers and photo/face data are not handled due to infrastructure limitations.
- High Accuracy:** 100% precision and recall on structured inputs, and over 90% coverage on complex clinical narratives.
- Real-Time Performance:** Average latency under 0.3 seconds per sample, meeting the requirements for interactive applications.
- Utility Preservation:** A 95% text utility score means the masked content remains meaningful and readable.
- Multi-LLM Support:** Successfully validated against OpenAI GPT-4o-mini, Mistral 7B, and FLAN-T5.
- Interactive Demo:** The Gradio front-end gives users a clear, side-by-side view of raw and masked outputs along with risk visualization.

5.2 Limitations

There are several constraints that must be resolved prior to wider deployment:

- Language Support:** The current implementation is English-only. Extending it to multilingual healthcare environments would require significant additional work.

- Contextual Ambiguity:** Some PHI detection depends on clinical context that regex patterns and NER models cannot reliably infer on their own.
- LLM Hallucinations:** The system cannot prevent a model from generating new, plausible-sounding PHI that was never present in the original input.
- Name Variants:** Non-Western naming conventions and informal name variations are not always caught consistently. Occurrences of same name does not get same hash.
- Adversarial Inputs:** Deliberately obfuscated PHI such as "John D0e" with zeros substituted for letters can bypass current detection.
- Custom Recognizer Activation:** The custom name_recognizer and email_recognizer defined in ProtectedHealthInformationRecognizers are currently disabled in the active pipeline (commented out in _init_presidio). The system falls back to Presidio's built-in SpaCy-based detection for PERSON and EMAIL_ADDRESS entities.

5.3 Future Work

Short-term improvements:

- Fine-tune a custom NER model on healthcare-specific PHI
- Add user-configurable confidence thresholds
- Support batch processing for large-scale EHR de-identification workflows
- Replace hardcoded API credentials (OpenAI key and Hugging Face token) in the notebook with environment variables or a secrets manager before any shared or production deployment

Medium-term enhancements:

- Add multilingual support (Spanish, Chinese, Arabic)
- Integrate differential privacy techniques
- Build a real-time streaming masker for chat applications

Long-term research directions:

- Explore adversarial training to reduce the risk of LLMs generating PHI
- Investigate federated learning for privacy-preserving model improvement
- Pursue formal verification of masking completeness

REFERENCES

- <https://microsoft.github.io/presidio/>
- <https://github.com/Privalyse/privalyse-mask>
- HIPAA Privacy Rule: Standards for Privacy of Individually Identifiable Health Information. 45 CFR § 160 and 164.
- Carlini, N., et al. (2021). Extracting Training Data from Large Language Models. USENIX Security Symposium.
- SpaCy: Industrial-strength Natural Language Processing. <https://spacy.io/>
- OpenAI. (2024). GPT-4o-mini: Efficient Language Models for Production.
- Hugging Face Transformers: State-of-the-art Machine Learning for JAX, PyTorch and TensorFlow.